

# Proiect

## Proiectarea Algoritmilor, Seria AC

# Knowledge Graph

---

## Introducere

Un **Knowledge Graph** este o structură de reprezentare a cunoștințelor în care entitățile din lumea reală (persoane, companii, locații, evenimente) și relațiile dintre ele sunt organizate sub forma unui graf. Astfel de sisteme stau la baza motoarelor de căutare semantică, a asistenților virtuali și a platformelor de recomandare.

În această temă veți implementa un astfel de sistem de la zero, în limbajul C. Veți parcurge întregul ciclu al unui sistem real: ingestia și indexarea datelor, construirea grafului și răspunsul eficient la interogări, inclusiv căutarea drumului de cost minim.

---

## Modelul de date

**Knowledge Graph-ul** este reprezentat printr-un **graf orientat ponderat**. Fiecare nod reprezintă o entitate, iar fiecare muchie orientată reprezintă o relație de la entitatea sursă la entitatea destinație, cu un cost asociat.

### Entități

O entitate este descrisă prin:

- `nume (char*)`, alocat dinamic;
- `tip (enum: PERSON, COMPANY, LOCATION, EVENT)`;
- `id (int)`, asignat în ordinea citirii, începând de la 0; unic per entitate.

### Relații

O relație este descrisă prin:

- entitatea sursă și entitatea destinație (referințe la nodurile din graf);
- `tip (enum: works_at, friend_of, located_in, participates_in)`;
- `cost (float)`, strict pozitiv.

Graful este **orientat**: relația Alice -> Google nu implică existența relației Google -> Alice.

## Interogări

Sistemul suportă cinci tipuri de interogări:

- EXISTS <entitate>: există entitatea în graf?
- EDGE <sursa> <destinație>: există o muchie directă între cele două entități?
- NEIGHBORS <entitate>: care sunt vecinii direcți ai entității?
- PATH <sursa> <destinație>: există un drum orientat între cele două entități?
- DIJKSTRA <sursa> <destinație>: care este drumul de cost minim?

## Pasul 1

(20p)

Ingestia datelor

Primul pas constă în citirea fișierelor de intrare și construirea imediată a structurilor de date care vor fi folosite pe tot parcursul temei. Spre deosebire de o abordare în două etape (citire → transformare), aici datele sunt integrate direct în structura finală pe măsura citirii, evitând duplicarea memoriei.

### 1.1. Lista de entități și graful (15p)

Citiți entitățile din fișierul `entitati.csv` (director `Pas_1`). Formatul fișierului este:

```
NUME;TIP
Alice;PERSON
Google;COMPANY
Prague;LOCATION
AI Summit;EVENT
```

Pe măsură ce citiți fiecare entitate, asigurați-i un id în ordine crescătoare și adăugați-o simultan în două structuri:

1. **O listă simplu înlănțuită**, care servește drept registru complet al tuturor entităților din sistem și va fi parcursă la afișarea grafului. Spre deosebire de o coadă (care implică consumarea elementelor prin extragere), lista permite acces repetat la toate entitățile fără a le distruge.
2. **Un nod în graful cu liste de adiacență**, fiecare entitate devenind imediat un nod al grafului, cu lista de muchii de ieșire inițial vidă. Construirea grafului direct în timpul citirii elimină necesitatea unei iterări suplimentare peste entități.

Apoi citiți relațiile din fișierul `relatii.csv`. Formatul este:

```
SURSA;RELATIE;DESTINATIE;COST
Alice;works_at;Google;2.50
Google;located_in;Prague;1.00
Alice;participates_in;AI Summit;3.00
AI Summit;located_in;Prague;1.50
```

Fiecare relație citită este adăugată **direct** în lista de adiacență a nodului sursă. Nu se mai stochează relațiile într-o listă intermediară separată, deoarece aceasta ar duplica memoria și ar introduce un pas de procesare inutil.

**Garanții ale fișierelor de intrare:** toate entitățile referite în `relatii.csv` există deja în `entitati.csv`; costurile sunt strict pozitive; nu există două entități cu același nume.

**Test 1.1** — Scrieți în `test_1_1.txt` (director `Pas_1`) lista de adiacență a grafului, în ordinea în care entitățile au fost citite. Costurile se afișează cu două zecimale.

Formatul fiecărei linii: `ID NUME: [ID_DEST TIP_RELATIE COST] ...`

Dacă un nod nu are muchii, se afișează doar numele nodului, fără lista de adiacență.

```
0 Alice: [1 works_at 2.50] [3 participates_in 3.00]
```

```
1 Google: [2 located_in 1.00]
2 Prague:
3 AI Summit: [2 located_in 1.50]
```

## 1.2. Coadă de interogări (5p)

Citiți interogările din fișierul `interogari.txt` și stocați-le într-o **coadă FIFO** implementată cu listă simplu înlănțuită.

O coadă este structura potrivită aici deoarece interogările trebuie procesate **în ordinea primirii** — exact semantica unui sistem real de tip query engine.

```
EXISTS Alice
EDGE Alice Google
NEIGHBORS Alice
PATH Alice Prague
DIJKSTRA Alice Prague
```

**Test 1.2** — Scrieți în `test_1_2.txt` (director `Pas_1`) interogările din coadă, în ordinea procesării (de la primul la ultimul intrat).

```
EXISTS Alice
EDGE Alice Google
NEIGHBORS Alice
PATH Alice Prague
DIJKSTRA Alice Prague
```

**Pasul 2**

(20p)

Indexarea pentru căutare eficientă

Interogările **EXISTS** și **EDGE** necesită găsirea unei entități după nume. O căutare liniară prin listă are complexitate  $O(n)$  per interogare — acceptabilă pentru grafuri mici, dar ineficientă pe seturi mari de date.

Soluția standard în sisteme reale este construirea unui **index** — o structură auxiliară care accelerează căutarea. Veți construi un **arbore binar de căutare (BST)** indexat după numele entităților, care reduce complexitatea căutării la  $O(\log n)$  în cazul mediu.

**2.1. Construirea BST-ului de indexare (10p)**

Parcurgeți lista de entități construită la Pasul 1 și inserați fiecare entitate în BST, folosind **numele** ca și cheie de comparație (comparație lexicografică, **strcmp**). Nodul BST nu duplică datele entității — stochează doar o **referință (pointer)** la nodul din graf.

**Observație:** un BST construit prin inserții în ordine aleatoare are înălțimea medie  $O(\log n)$ . Dacă entitățile sunt citite în ordine alfabetică, BST-ul degenerază într-o listă ( $O(n)$ ). Pentru acest exercițiu se garantează că fișierele de test nu sunt sortate alfabetic, deci nu este necesară o variantă auto-balansată (AVL/Red-Black).

**Test 2.1** — Scrieți în `test_2_1.txt` (director `Pas_2`) rezultatul parcurgerii **în inordine** (stânga-rădăcină-dreapta) a BST-ului. Parcurgerea în inordine a unui BST produce elementele în ordine lexicografică; acesta este testul natural că BST-ul a fost construit corect.

Afișați pe fiecare linie: ID NUME TIP

```
3 AI Summit EVENT
0 Alice PERSON
1 Google COMPANY
2 Prague LOCATION
```

**2.2. Interogări de bază folosind indexul (10p)**

Procesați interogările de tip **EXISTS**, **EDGE** și **NEIGHBORS** din coada construită la Pasul 1, în ordinea primirii.

- **EXISTS X**: căutați **X** în BST, complexitate  $O(\log n)$ . Dacă există, răspundeți **DA**, altfel **NU**.
- **EDGE X Y**: căutați **X** în BST, apoi parcurgeți lista de adiacență a nodului găsit pentru a verifica existența muchiei către **Y**, complexitate  $O(\log n + \text{grad}(X))$ .
- **NEIGHBORS X**: căutați **X** în BST, apoi afișați toți vecinii direcți în ordinea din lista de adiacență. Dacă entitatea nu există sau nu are vecini, afișați **NULL**.

Interogările PATH și DIJKSTRA rămân în coadă și vor fi procesate la Pasul 3.

**Test 2.2** — Scrieți în `test_2_2.txt` (director `Pas_2`) rezultatele interogărilor EXISTS, EDGE și NEIGHBORS, în ordinea extragerii din coadă:

```
EXISTS Alice: DA
EDGE Alice Google: DA
NEIGHBORS Alice: Google AI Summit
```

## Pasul 3

(20p)

Căutarea drumurilor

Interogările de tip **PATH** și **DIJKSTRA** necesită traversarea grafului dincolo de vecinii direcți ai unui nod. Cele două operații răspund la întrebări diferite: **PATH** verifică existența oricărui drum, iar **DIJKSTRA** găsește drumul de cost minim.

### 3.1. Existența drumurilor cu BFS (10p)

Pentru fiecare interogare **PATH X Y** rămasă în coadă, determinați dacă există un drum orientat de la **X** la **Y** folosind **BFS (Breadth-First Search)**.

BFS explorează graful nivel cu nivel, folosind intern o **coadă** pentru a ține evidența nodurilor de explorat; structură pe care ați implementat-o deja la Pasul 1. BFS garantează că drumul găsit are numărul minim de muchii (nu neapărat costul minim). Marcați nodurile vizitate pentru a evita ciclurile.

Dacă drumul există, reconstruiți și afișați calea parcurgând vectorul de părinți reținut în timpul BFS-ului.

**Test 3.1** — Scrieți în `test_3_1.txt` (director `Pas_3`) rezultatele interogărilor **PATH**, în ordinea extragerii din coadă:

- dacă există drum: **PATH X Y: X -> nod1 -> ... -> Y**
- dacă nu există drum: **PATH X Y: NU**

```
PATH Alice Prague: Alice -> Google -> Prague
```

### 3.2. Drumul de cost minim cu Dijkstra (10p)

Pentru fiecare interogare **DIJKSTRA X Y** rămasă în coadă, determinați drumul de cost minim de la **X** la **Y** folosind **algoritmul Dijkstra**. Costul total al unui drum este suma costurilor tuturor muchiilor componente.

La fiecare pas, Dijkstra selectează nodul neprocesat cu costul curent minim. Eficiența acestei selecții depinde direct de structura de date folosită: o căutare liniară duce la  $O(n^2)$ , în timp ce un **min-heap** reduce complexitatea la  $O((n+m) \log n)$ , unde  $n$  este numărul de noduri și  $m$  numărul de muchii.

**Cerință de eficiență:** implementați selecția minimului folosind un **min-heap explicit**. Motivați alegerea în documentație și calculați complexitatea algoritmului. Implementările cu căutare liniară vor fi depunctate.

**Test 3.2** — Scrieți în `test_3_2.txt` (director `Pas_3`) rezultatele interogărilor **DIJKSTRA**, în ordinea extragerii din coadă. Costul se afișează cu două zecimale. Dacă există mai multe drumuri cu același cost minim, poate fi afișat oricare.

- dacă există drum: **DIJKSTRA X Y: COST = val; DRUM = X -> ... -> Y**

- dacă nu există drum: DIJKSTRA X Y: NU

```
DIJKSTRA Alice Prague: COST = 3.50; DRUM = Alice -> Google -> Prague
```



## Pasul 4

(20p)

Procesarea completă a interogărilor

Procesați **toate** interogările din coadă, indiferent de tip, și scrieți rezultatele într-un fișier centralizat, în ordinea primirii. Acest pas combină toate operațiile implementate anterior.

La extragerea fiecărei interogări din coadă, determinați tipul acesteia și apelați funcția corespunzătoare: căutare în BST pentru **EXISTS** și **EDGE**, parcurgere a listei de adiacență pentru **NEIGHBORS**, BFS pentru **PATH**, și Dijkstra pentru **DIJKSTRA**.

**Test 4** — Scrieți în `test_4.txt` (director `Pas_4`) toate rezultatele, în ordinea extragerii din coadă, folosind formatele definite la pașii anteriori.

```
EXISTS Alice: DA
EDGE Alice Google: DA
NEIGHBORS Alice: Google AI Summit
PATH Alice Prague: Alice -> Google -> Prague
DIJKSTRA Alice Prague: COST = 3.50; DRUM = Alice -> Google -> Prague
```

## Documentație

(10p)

Completați fișierul `README.md` cu o descriere concisă a implementării. Nu este necesar să documentați fiecare funcție. Includeți cel puțin:

- structurile de date utilizate și motivul pentru care fiecare a fost aleasă (lista vs. coadă vs. BST vs. liste de adiacență vs. min-heap);
- complexitatea operațiilor **EXISTS**, **EDGE**, **NEIGHBORS** în funcție de  $n$  (număr de noduri) și  $m$  (număr de muchii);
- complexitatea algoritmilor **PATH** (BFS) și **DIJKSTRA**, cu justificarea structurii heap-ului ales.

## Cerințe tehnice

Pe lângă punctajele menționate, la nota finală vor fi luate în considerare prezentarea și răspunsurile la întrebări, precum și următoarele aspecte:

- memoria se alocă **dinamic**; nu se acceptă buffere statice de dimensiune fixă pentru date de dimensiune variabilă (ex.: câmpul `nume`);
- fiecare funcție are **cel mult 40 de linii**; operațiile pe structuri de date (inserție, căutare, ștergere) se implementează în funcții separate, cu nume sugestive;
- se rezolvă **toate warning-urile de compilare**; un program cu warning-uri poate funcționa local dar poate produce comportament nedefinit pe checker;
- **soluția este punctată strict** pe baza rezultatelor checkerului;

- soluțiile cu complexitate nejustificat de mare vor fi depunctate;
- temele sunt verificate **anti-plagiat**; ambele persoane implicate într-un caz de copiere vor fi depunctate.

## Recomandare metodologică

Dezvoltați soluția **incremental**: implementați și testați complet fiecare pas înainte de a trece la următorul. Construiți exemple mici de intrare și verificați manual ieșirile înainte de a rula checker-ul. O eroare în Pasul 1 (ex.: id-uri greșite, muchii lipsă) se va propaga și va compromite toți pașii următori.

## Referințe relevante

1. Thomas H. Cormen et al. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
2. GeeksforGeeks. Dijkstra's Shortest Path Algorithm.  
<https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-greedy-algo-7/>
3. GeeksforGeeks. Binary Search Tree.  
<https://www.geeksforgeeks.org/binary-search-tree-data-structure/>